

Pipeline Lab Manual

Overview — Ten Jenkins Laboratories

Ten laboratory sessions built entirely hands-on inside Jenkins — administering the controller, writing Jenkinsfiles, wiring up GitHub, gating on tests and security scans, provisioning infrastructure as code, running dynamic Kubernetes agents, and orchestrating a real deployment.

Each lab builds on the last against one running case study, **Taskflow**: a small task-tracking API (`taskflow-api` , Node.js 20 / Express / PostgreSQL) and, from Lab 10 onward, its Flutter client (`taskflow-mobile`). Fork both repositories at the start of Lab 01 — every later lab assumes they already exist in your GitHub account.

Labs	10
Contact time	~34 hours
Core platform	Jenkins
Supporting stack	Docker · Kubernetes · Terraform · Prometheus

LAB-TO-LECTURE MAP

Lab	Title	Lecture deck	Core tools	Hours
01	Installing Jenkins & the First Job	04–05 — Jenkins intro / install	Docker, Jenkins	3 h
02	Jenkins Plugins, Global Tools & RBAC	04–05, 11 — Jenkins intro / Best practices	Plugin Manager, Role Strategy	2 h
03	Your First Declarative Pipeline	03, 06, 07 — Stages / Jenkinsfile / Examples	Jenkinsfile, Groovy	3 h
04	Git, GitHub & Multibranch Pipelines	08 — Git/GitHub integration	GitHub webhooks, Jenkins	3 h
05	Automated Testing & Quality Gates	09 — Automated testing	Jest, Playwright, SonarQube	4 h
06	Shift-Left Security Pipeline	00c, 09b — DevSecOps / Security	Gitleaks, Semgrep, npm audit, Syft	4 h
07	Containers, Image Scanning & Deploy	07, 09b, 10 — Examples / Security / Deploy	Docker, Trivy, kubectl	4 h
08	Infrastructure as Code in the Pipeline	00d — Infrastructure as Code	Terraform, Ansible, tfsec/Checkov	4 h
09	Jenkins on Kubernetes: Dynamic Agents & Metrics	04, 00e — Jenkins architecture / SRE	Kubernetes plugin, Prometheus, Grafana	4 h
10	Capstone: End-to-End Pipeline	11–13 — Best practices / Summary / Mobile	Everything above + Flutter/Gradle	4 h

The Taskflow case study. Labs 1–10 all operate on the same two repositories: `taskflow-api` and, from Lab 10 onward, `taskflow-mobile`. Fork both at the start of Lab 01 — every later lab assumes they already exist in your GitHub account.

HOW TO USE THIS MANUAL

Each lab follows the same structure: **Objectives**, **Background**, **Environment**, **Tasks** (numbered, do them in order), **Deliverables** (what to hand in), and **Assessment** (a 100-point rubric). Labs are designed for one 3–4 hour session each, individually or in pairs as noted, except the Lab 10 capstone, which runs as a team of 3–4 with take-home time for the mobile build.

Lab 01 — Installing Jenkins & the First Job

Maps to Deck 04–05 — Introduction to Jenkins / Installing Jenkins

Duration: 3 h · **Mode:** Individual · **Deliverable:** Screenshots + running container

OBJECTIVES

- Stand up a Jenkins controller in Docker and complete first-run setup.
- Explain the controller/agent/executor architecture and configure a second agent.
- Create and run a Freestyle job that checks out `taskflow-api`.

BACKGROUND

Jenkins separates the **controller** (schedules builds, serves the UI) from **agents** (execute the actual steps) so that heavy builds never block the controller. Understanding this split now makes the `agent` directive in every later Jenkinsfile self-explanatory instead of magic.

ENVIRONMENT

- Docker Desktop or Docker Engine, 2 GB RAM free.
- A GitHub account with your fork of `taskflow-api`.

TASKS

1. Fork `taskflow-api` and `taskflow-mobile` into your own GitHub account. Keep them public or add a Jenkins-readable deploy key.

2. Launch Jenkins:

```
# named volume persists jenkins_home across restarts
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts-jdk21
```

3. Retrieve the initial admin password and complete setup with the **suggested plugins** set:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

4. Under **Manage Jenkins → Nodes**, add a second permanent agent labeled `linux-build` (a plain Docker container running the `jenkins/inbound-agent` image is sufficient). Confirm it connects and shows 1 executor.
5. Create a **Freestyle project** named `taskflow-smoke` that: checks out your fork, runs `npm ci`, and echoes the Node version. Pin it to run on the `linux-build` label.
6. Trigger a build manually and confirm it succeeds; open the console log and identify which agent executed it.

DELIVERABLES

- Screenshot of **Manage Jenkins** → **Nodes** showing both the built-in and `linux-build` agents online.
- Console output of a green `taskflow-smoke` build.
- One paragraph explaining, in your own words, what would happen to running builds if the controller container were restarted mid-build.

ASSESSMENT

Criterion	Points
Jenkins reachable with two functioning nodes	35
Freestyle job builds green on the labeled agent	35
Correct explanation of controller/agent failure behavior	30

Lab 02 — Jenkins Plugins, Global Tools & RBAC

Maps to Deck 04–05, 11 — Introduction to Jenkins / Best Practices

Duration: 2 h · **Mode:** Individual · **Deliverable:** Roles matrix + backup archive

OBJECTIVES

- Install and configure the plugin set a real pipeline depends on, beyond the “suggested” defaults.
- Configure Global Tool auto-installers so a Jenkinsfile never depends on a hand-provisioned agent.
- Replace Jenkins’ default open security realm with least-privilege, role-based access control.

BACKGROUND

Jenkins ships thin and grows through its 1800+ plugin ecosystem — the essential set (Git, Docker Pipeline, Blue Ocean, SonarQube Scanner, Kubernetes, Credentials Binding) is what turns a bare controller into the platform every later lab assumes exists. Left on its defaults, a fresh Jenkins also gives every logged-in user full admin rights — the first best-practice fix, before any pipeline work, is locking that down.

TASKS

1. From **Manage Jenkins → Plugins**, install: Docker Pipeline, Blue Ocean, SonarQube Scanner, Kubernetes, Credentials Binding, and Slack Notification — the essential set beyond the suggested defaults.
2. Under **Global Tool Configuration**, add a NodeJS auto-installer (`node20`) and a JDK auto-installer (`temurin-21`). Add a `tools { nodejs 'node20' }` block to a scratch Jenkinsfile and confirm it provisions Node without a Docker agent.
3. Install the **Role-based Authorization Strategy** plugin. Disable anonymous read access, then define two roles: `developer` (build and read permission on jobs matching `taskflow-.*`) and `admin` (full control). Assign yourself `admin` and a lab partner `developer` .
4. Create a folder-scoped **Secret text** credential. Log in as the `developer` account and confirm the UI lets you **reference** the credential ID in a Jenkinsfile but never displays its value.
5. Open **Blue Ocean** and run the `taskflow-smoke` job from Lab 01 through it; confirm the visual pipeline graph matches the stages you expect.
6. Back up `JENKINS_HOME` :

```
docker run --rm --volumes-from jenkins \
-v $(pwd):/backup alpine \
tar czf /backup/jenkins_home.tgz /var/jenkins_home
```

Write the three-line restore procedure and verify it by restoring into a second, throwaway Jenkins container.

DELIVERABLES

- Screenshot of the installed plugin list and the two configured Global Tool auto-installers.
- Screenshot of the roles matrix and proof the `developer` account cannot read the credential's value.
- `jenkins_home.tgz` plus a verified restore procedure, demonstrated on a second container.

ASSESSMENT

Criterion	Points
All required plugins installed and the tool auto-installers work	25
RBAC correctly restricts the <code>developer</code> role; anonymous access disabled	30
Credential is usable by reference but never exposed to a non-admin	25
Backup restored successfully into a second Jenkins instance	20

Lab 03 — Your First Declarative Pipeline

Maps to Deck 03, 06, 07 — Pipeline Stages / Jenkinsfile / Examples

Duration: 3 h · **Mode:** Individual · **Deliverable:** Jenkinsfile in repo

OBJECTIVES

- Write a `Jenkinsfile` with agent, environment, stages, and post blocks from scratch.
- Run a build inside a Docker agent instead of the controller's shell.
- Use `post` conditions to react differently to success and failure.

BACKGROUND

A Freestyle job lives only inside Jenkins' database. A `Jenkinsfile` lives in the repository next to the code it builds — it is reviewed in pull requests exactly like application code, which is the entire point of Pipeline as Code.

TASKS

1. In the root of `taskflow-api`, add a `Jenkinsfile` that runs in a `node:20-alpine` agent, with stages `Install` (`npm ci`), `Lint` (`npm run lint`) and `Unit Test` (`npm test`).
2. Add an `environment` block defining `APP_NAME` and `NODE_ENV=test`, and reference both inside an `echo` step.
3. Add `options { timeout(time: 10, unit: 'MINUTES') }` and explain in a code comment why a pipeline stage should never run unbounded.
4. Add a `post` block: `success` echoes a green check message; `failure` echoes the failing stage name via `env.STAGE_NAME`; `always` archives `npm-debug.log*` if present.
5. Create a Jenkins **Pipeline** job (not Freestyle) pointing at `Jenkinsfile` in your fork's `main` branch, via Pipeline script from SCM.
6. Deliberately break a test, run the build, confirm the `failure` post-block fires, then fix it and confirm `success` fires.

A minimal starting point:

```

pipeline {
  agent { docker { image 'node:20-alpine' } }

  environment {
    APP_NAME = 'taskflow-api'
    NODE_ENV = 'test'
  }

  options {
    timeout(time: 10, unit: 'MINUTES')
    // A hung npm install or test run must not hold the executor forever
  }

  stages {
    stage('Install') { steps { sh 'npm ci' } }
    stage('Lint')    { steps { sh 'npm run lint' } }
    stage('Unit Test') { steps { sh 'npm test' } }
  }

  post {
    success { echo "    ${env.APP_NAME} passed on ${env.NODE_ENV}" }
    failure { echo "    Failed at stage: ${env.STAGE_NAME}" }
    always  { archiveArtifacts artifacts: 'npm-debug.log*', allowEmptyArchive: true }
  }
}

```

DELIVERABLES

- Merged `Jenkinsfile` in your fork's `main` branch.
- Two console logs: one red build, one green build, both showing the correct `post` message.

ASSESSMENT

Criterion	Points
Jenkinsfile runs correctly inside a Docker agent	30
environment variables used and interpolated correctly	15
post blocks each fire under the correct condition	35
Timeout justification comment is technically sound	20

Lab 04 — Git, GitHub & Multibranch Pipelines

Maps to Deck 08 — Integrating with Git & GitHub

Duration: 3 h · **Mode:** Individual · **Deliverable:** Live webhook + branch matrix

OBJECTIVES

- Wire a GitHub webhook so pushes trigger Jenkins automatically.
- Configure a Multibranch Pipeline that discovers branches and pull requests.
- Gate a `Deploy` stage with a `when { branch }` condition matching a real branch strategy.

BACKGROUND

Real teams don't click "Build Now" — a push does it for them. A Multibranch Pipeline job scans the repository and gives every branch and PR its own pipeline run, each obeying the same Jenkinsfile but taking different paths through `when` conditions.

TASKS

1. Expose your local Jenkins to the internet for webhook delivery (use a tunnel such as `ngrok` if Jenkins is not otherwise reachable), and register a webhook in `taskflow-api` → **Settings** → **Webhooks** pointed at `<jenkins-url>/github-webhook/` for `push` and `pull_request` events.
2. Create a **Multibranch Pipeline** job pointed at your GitHub fork; confirm it auto-discovers `main`, `develop`, and any open PR branches.
3. Create branches `develop` and `feature/health-endpoint`. Extend the Jenkinsfile's stages:

```
stage('Deploy - Staging') {
    when { branch 'develop' }
    steps { sh 'echo deploying to staging...' }
}

stage('Deploy - Production') {
    when { branch 'main' }
    input { message 'Deploy to production?' }
    steps { sh 'echo deploying to production...' }
}
```

4. Push a commit to `feature/health-endpoint` and confirm the webhook triggers a build automatically, and that neither deploy stage runs.
5. Open a pull request from `feature/health-endpoint` into `develop` and confirm Jenkins builds the PR itself as a distinct job.
6. Merge to `develop`, confirm `Deploy - Staging` fires without approval; merge `develop` into `main`, confirm `Deploy - Production` pauses on the `input` step until you approve it.

DELIVERABLES

- Screenshot of the GitHub webhook delivery log showing a `200` response.
- A one-diagram branch strategy (feature → develop → main) annotated with which stage runs on each.

ASSESSMENT

Criterion	Points
Webhook triggers builds without manual intervention	25
Multibranch job correctly discovers branches and PRs	25
<code>when</code> conditions correctly gate each deploy stage	30
Production <code>input</code> gate behaves as designed	20

Lab 05 — Automated Testing & Quality Gates

Maps to Deck 09 — Automated Testing in Jenkins

Duration: 4 h · **Mode:** Pairs · **Deliverable:** Coverage + quality gate report

OBJECTIVES

- Publish JUnit-format results and Cobertura coverage from the pipeline.
- Run a Playwright end-to-end suite against a Dockerized instance of `taskflow-api`.
- Enforce a SonarQube quality gate that blocks the pipeline below a coverage threshold.

BACKGROUND

A test that runs but whose result nobody can see does not make a pipeline safer. `junit` and `publishCoverage` turn raw test output into a trend Jenkins can graph and gate on — and a quality gate turns “tests passed” into “tests passed **and** the code meets our bar.”

TASKS

1. Add a `Unit Test` stage running `npm test -- --coverage --reporters=jest-junit`; in `post.always`, call `junit 'reports/junit.xml'` and `publishCoverage` adapters: `[coberturaAdapter('coverage/cobertura-coverage.xml')]`.
2. Stand up a local SonarQube (`docker run -d -p 9000:9000 sonarqube:lts-community`), create a project token, and store it as a Jenkins `Secret text` credential named `sonar-token`.
3. Add a `SonarQube Analysis` stage using `withSonarQubeEnv` and `sonar-scanner`, followed by a `Quality Gate` stage that calls `waitForQualityGate abortPipeline: true` inside a 5-minute timeout:

```
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('SonarQube') {
      sh 'sonar-scanner -Dsonar.projectKey=taskflow-api'
    }
  }
}
stage('Quality Gate') {
  steps {
    timeout(time: 5, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true
    }
  }
}
```

4. In SonarQube, set the quality gate's coverage condition to **fail below 70%**. Deliberately strip enough tests to drop under 70% and confirm the pipeline aborts at the gate, not at the test stage.

5. Add an `E2E` stage using the `mcr.microsoft.com/playwright` Docker agent that starts the API with `docker compose up -d`, runs at least three Playwright specs (list tasks, create task, mark task done) against it, and publishes the JUnit and HTML reports.
6. Restore full test coverage, re-run, and confirm all three gates (unit tests, quality gate, E2E) go green in one pipeline execution.

DELIVERABLES

- The Jenkins build's test-trend graph showing at least two builds (one red at the gate, one green).
- Exported SonarQube quality gate report (PDF or screenshot) for the passing build.
- Playwright HTML report artifact from Jenkins.

ASSESSMENT

Criterion	Points
JUnit + coverage correctly published and trending	25
Quality gate genuinely blocks a real regression	30
Playwright E2E suite runs headless in CI and reports correctly	30
Pipeline is fully green end-to-end after the fix	15

Lab 06 — Shift-Left Security Pipeline

Maps to Deck 00c, 09b — DevSecOps / Security Analysis

Duration: 4 h · **Mode:** Pairs · **Deliverable:** Signed SBOM + scan reports

OBJECTIVES

- Run secrets detection, SAST and SCA stages before the build stage, in that order.
- Generate and archive an SBOM for a release artifact.
- Apply a fail/warn threshold policy instead of an all-or-nothing gate.

BACKGROUND

Shift-left means the cost curve in the DevSecOps lecture: a leaked secret caught by Gitleaks at commit time costs nothing; the same secret found in production costs an incident. This lab builds the ordered security stage chain — **Secrets** → **SAST** → **SCA** → **Container** → **DAST** — for the Node.js stack, stopping short of DAST (covered implicitly once the app is deployed in Lab 07).

TASKS

1. Add a **Secrets Detection** stage running Gitleaks against the full repo history; intentionally commit a fake AWS key on a scratch branch first and confirm Gitleaks catches it before proceeding.
2. Add an **SAST** stage running `npx eslint --plugin security src/` and a Semgrep stage with `--config=p/owasp-top-ten --config=p/nodejs`, archiving SARIF output from both.
3. Add an **SCA** stage running `npm audit --audit-level=high --json`; wrap it in a script block that parses the JSON and fails the build only if **critical** count > 0, warns otherwise (fail/warn threshold, not exit-code-only):

```
stage('SCA - npm audit') {
  steps {
    script {
      sh 'npm audit --audit-level=high --json > audit.json || true'
      def critical = sh(
        script: "jq '.metadata.vulnerabilities.critical' audit.json",
        returnStdout: true
      ).trim().toInteger()
      if (critical > 0) {
        error("Blocking: ${critical} critical vulnerabilities found")
      }
      echo "SCA passed with 0 critical vulnerabilities (warnings allowed)"
    }
  }
}
```

4. Add a **Generate SBOM** stage using Syft to emit a CycloneDX SBOM for `taskflow-api`, then sign it with Cosign (a local keypair is fine for the lab) and archive both the SBOM and its signature.

5. Write a two-clause OPA/Rego policy (in `policy/security.rego`) that denies any build where a dependency scan reports a `CRITICAL` CVE, and add a **Policy Gate** stage that evaluates it with `opa eval`.
6. Deliberately downgrade one dependency to a version with a known CVE, confirm the Policy Gate blocks the pipeline, then upgrade it back and confirm the pipeline passes.

DELIVERABLES

- Archived Gitleaks report showing the caught test secret (from the scratch branch, not merged).
- Signed SBOM (`.cdx.json` + signature) as a build artifact.
- The `policy/security.rego` file and one build log showing it blocking the CVE, one showing it passing.

ASSESSMENT

Criterion	Points
Correct stage order: secrets → SAST → SCA → SBOM → policy	25
Fail/warn threshold logic is correct, not a blanket exit-zero	25
SBOM generated, signed, and archived	25
Policy gate demonstrably blocks and un-blocks correctly	25

Lab 07 — Containers, Image Scanning & Deployment

Maps to Deck 07, 09b, 10 — Jenkinsfile Examples / Security / Deploying

Duration: 4 h · **Mode:** Pairs · **Deliverable:** Running blue/green service

OBJECTIVES

- Build and push a versioned Docker image from the pipeline, never using the `latest` tag.
- Scan the built image with Trivy and block on high/critical CVEs.
- Implement a blue/green deployment against a local Kubernetes cluster (kind or minikube).

BACKGROUND

Everything up to this lab produced test reports. This lab produces the thing users actually run. Blue/green deployment lets you switch a Kubernetes Service's selector between two live deployments instead of mutating one in place — the rollback is a label change, not a redeploy.

TASKS

1. Add a **Build Image** stage tagging the image `taskflow-api:${env.GIT_COMMIT.take(7)}` — never `latest` — and pushing it to a local registry (`registry:2` container is fine).
2. Add a **Container Scan** stage running Trivy with `--exit-code 1 --severity HIGH,CRITICAL` against the freshly built tag; archive the SARIF report regardless of outcome.
3. Stand up a local Kubernetes cluster (`kind create cluster`) with two Deployments, `taskflow-blue` and `taskflow-green` , and a single Service whose selector picks one color via a label.
4. Write a **Blue/Green Deploy** stage:

```

stage('Blue/Green Deploy') {
    steps {
        script {
            def current = sh(
                script: "kubectl get svc taskflow -o jsonpath='{.spec.selector.color}'",
                returnStdout: true
            ).trim()
            def next = current == 'blue' ? 'green' : 'blue'

            sh "kubectl set image deployment/taskflow-${next} app=taskflow-api:${
                env.GIT_COMMIT.take(7)}"
            sh "kubectl rollout status deployment/taskflow-${next}"

            // smoke test the new pods directly, bypassing the Service
            sh "kubectl run smoke-${BUILD_NUMBER} --rm -i --restart=Never --image=curlimages/
curl -- \
            curl -sf http://taskflow-${next}:8080/health"

            sh "kubectl patch svc taskflow -p '{\"spec\":{\"selector\":{\"color\":\"${next}
\"]]}'"
            echo "Switched traffic from ${current} to ${next}"
        }
    }
}

```

5. Add a `post.failure` block that patches the Service selector back to the previous color — an automated rollback, not a manual one.
6. Demonstrate a full cycle twice: once where the new version passes its smoke test and traffic switches, once where you inject a broken image and confirm the automatic rollback keeps the old color serving traffic.

DELIVERABLES

- `kubectl get svc taskflow -o yaml` output before and after a successful switch.
- Trivy SARIF report for one image build.
- Console log of the failed deploy showing the automatic rollback firing.

ASSESSMENT

Criterion	Points
Images are immutably tagged and pushed correctly	20
Trivy gate genuinely blocks a vulnerable image	25
Blue/green switch works and is verified by smoke test first	30
Automatic rollback on failure is demonstrated, not just coded	25

Lab 08 — Infrastructure as Code in the Pipeline

Maps to Deck 00d — Infrastructure as Code

Duration: 4 h · **Mode:** Pairs · **Deliverable:** Applied Terraform state

OBJECTIVES

- Write Terraform to provision the environment `taskflow-api` deploys into, with remote state.
- Lint and security-scan IaC before every `plan`, and gate `apply` behind human approval.
- Use Ansible to configure a provisioned host after Terraform creates it.

BACKGROUND

Lab 07 assumed a Kubernetes cluster already existed. This lab builds the infrastructure itself as version-controlled, declarative configuration — the difference between “someone clicked buttons in a console” and “every environment change is a reviewable diff.”

TASKS

1. Write Terraform (against LocalStack or a real free-tier cloud account — instructor’s choice) that provisions: one compute instance, one security group allowing port 8080, and outputs the instance’s address. Configure an S3-compatible (or LocalStack) backend for remote state — never commit `terraform.tfstate`.
2. Add an **IaC Lint & Validate** stage running `terraform fmt -check -recursive` and `terraform validate` in parallel with `ansible-lint` on a companion playbook:

```
stage('IaC Lint & Validate') {
    parallel {
        stage('Terraform Validate') {
            steps {
                dir('infra/terraform') {
                    sh 'terraform init -backend=false'
                    sh 'terraform validate'
                    sh 'terraform fmt -check -recursive'
                }
            }
        }
        stage('Ansible Lint') {
            steps { sh 'ansible-lint infra/ansible/playbook.yml' }
        }
    }
}
```

3. Add an **IaC Security Scan** stage running `tfsec` and `checkov` against the Terraform directory; fix at least one real finding each tool reports (e.g., an open security group or missing encryption flag) before proceeding.

4. Add a **Terraform Plan** stage that archives the plan file, followed by an **Approval** stage (`input`) that shows the plan summary before **Terraform Apply** runs — apply must never happen unattended in this lab.
5. Write an Ansible playbook that installs Node.js and Docker on the provisioned host and pulls the `taskflow-api` image built in Lab 07. Add a **Configure with Ansible** stage that builds a dynamic inventory from the Terraform output and runs the playbook.
6. Tear the environment down with `terraform destroy` at the end of the session and confirm the state file reflects zero managed resources.

DELIVERABLES

- Terraform and Ansible source files, plus the archived `tfplan` artifact from Jenkins.
- Before/after tfsec or Checkov output showing the fixed finding.
- Screenshot of the approval prompt and the applied output showing the instance's address.

ASSESSMENT

Criterion	Points
Remote state configured correctly; no local state committed	20
tfsec/Checkov findings genuinely triaged and fixed	25
Apply is gated behind a real human approval step	20
Ansible playbook successfully configures the provisioned host	25
Clean <code>destroy</code> with no orphaned resources	10

Lab 09 — Jenkins on Kubernetes: Dynamic Agents & Pipeline Metrics

Maps to Deck 04, 00e — Jenkins Architecture / Site Reliability Engineering

Duration: 4 h · **Mode:** Pairs · **Deliverable:** Dashboard + fired alert

OBJECTIVES

- Run Jenkins build agents as ephemeral Kubernetes pods instead of long-lived static containers.
- Expose Jenkins' own build and queue metrics to Prometheus and chart them in Grafana.
- Define an SLO for the pipeline itself and alert on it going into the red, live.

BACKGROUND

Every earlier lab pinned `agent { docker { image ... } }` to a container that sat around between builds. A Kubernetes agent instead requests a fresh pod per run and tears it down afterward — the same elastic-capacity idea from the IaC lecture's Helm/Kubernetes material, applied to the build farm itself. And a pipeline is a service like any other: it has a queue, a success rate, and a latency — which means SRE's SLI/SLO/error-budget machinery from the lecture applies directly to Jenkins, not just to `taskflow-api`.

TASKS

1. Install the **Kubernetes** plugin and configure a cloud pointed at a local `kind` cluster; define a Pod Template offering a `node:20-alpine` container labeled `k8s-node`.
2. Rewrite `taskflow-api`'s `agent` block to use a Kubernetes pod instead of the static Docker agent from Lab 03:

```
agent {
  kubernetes {
    yaml '''
    apiVersion: v1
    kind: Pod
    spec:
      containers:
      - name: node
        image: node:20-alpine
        command: ['cat']
        tty: true
    '''
  }
}
```

Trigger a build and use `kubectl get pods -w` to watch a real pod appear and terminate for that single run.

3. Install the Jenkins **Prometheus metrics** plugin; confirm `<jenkins-url>/prometheus` exposes build duration histograms, queue length, and executor counts.

4. Add Jenkins as a Prometheus scrape target and build a Grafana dashboard with three panels: build success rate, p95 build duration, and current queue length.
5. Define the pipeline's own SLO — **95% of builds complete in under 6 minutes, measured over a rolling 7 days** — and write a Prometheus alerting rule that fires on **queue wait time > 2 minutes for 5 minutes straight** (a symptom of insufficient capacity, not a raw resource cause):

```
groups:
- name: jenkins-slo
  rules:
  - alert: JenkinsQueueBacklog
    expr: jenkins_queue_size_value > 0 and
      avg_over_time(jenkins_queue_size_value[5m]) > 0
    for: 5m
    labels: { severity: warning }
    annotations:
      summary: "Jenkins build queue backlog exceeds 2 minutes"
```

6. Saturate Jenkins deliberately: trigger 10 concurrent builds against a cloud configured for only 2 concurrent pods, confirm the queue-time alert fires, then raise the Pod Template's concurrency limit and confirm queue time — and the alert — recover.

DELIVERABLES

- Diff of the Jenkinsfile showing the Docker agent replaced by a Kubernetes pod template.
- Exported Grafana dashboard JSON for the three pipeline-health panels.
- Screenshots of the queue-time alert firing under load and clearing after the capacity fix.

ASSESSMENT

Criterion	Points
Builds genuinely run on ephemeral Kubernetes pods, confirmed live	30
Jenkins Prometheus metrics correctly scraped and dashboarded	25
SLO and alert rule are symptom-based and technically correct	20
Saturation and recovery both demonstrated, not just described	25

Lab 10 — Capstone: End-to-End Pipeline

Maps to Deck 11–13 — Best Practices / Summary / Dart-Android Pipeline

Duration: 4 h + take-home · **Mode:** Team of 3–4 · **Deliverable:** One unified pipeline, demoed live

OBJECTIVES

- Compose every stage from Labs 03–09 into a single, parallelized, well-organized Jenkinsfile.
- Extend the pipeline to also build and sign the `taskflow-mobile` Flutter client.
- Present the pipeline live: trigger a real change, walk the reviewers through every gate as it fires.

BACKGROUND

This is the lecture's own closing diagram made real:

`Code → Commit → Build → Test → Stage → Deploy → Monitor`, one Jenkinsfile, for two coordinated repositories. Nothing here is new engineering — it is disciplined integration and organization of Labs 3 through 9.

TASKS

1. Restructure `taskflow-api`'s Jenkinsfile so independent stages run in `parallel` blocks (lint, unit tests, SAST, SCA together) per the fail-fast/parallelize best practice, while dependent stages (build → scan → deploy) stay sequential.
2. Ensure every secret (registry credentials, Sonar token, kubeconfig, Android keystore) is bound via `withCredentials` or `credentials()` — a code review checklist item is that `grep -R "password\\|secret\\|token" Jenkinsfile` finds no literal values.
3. Add the Android/Flutter side: a `Jenkinsfile` for `taskflow-mobile` using the `cirruslabs/flutter:stable` image with stages `flutter analyze`, `flutter test --coverage`, an `osv-scanner` SCA pass, a debug APK build on every branch, and a signed release AAB build (via a Jenkins-stored keystore) gated to `main`.
4. Move both Jenkinsfiles' build/test stages onto the Kubernetes dynamic agents configured in Lab 09 — no static, hand-provisioned containers left anywhere in either pipeline — and add a **Pipeline Health Gate** stage before `Deploy – Production` that queries the Lab 09 Prometheus endpoint and aborts the deploy if the rolling build success rate over the last 20 builds is below 90%.
5. Add Slack (or email) notifications on `success` and `failure` for both pipelines, including the branch name and build URL.
6. Produce a one-page architecture diagram of the full pipeline (both Jenkinsfiles, all gates, in order) and a rollback runbook: the exact steps an on-call engineer follows if the production deploy stage fails.

7. **Live demo:** push a real, reviewer-assigned change (e.g., a new API field surfaced in the mobile UI) and narrate every stage as it executes, including at least one gate that the team demonstrates blocking a bad change live.

DELIVERABLES

- Both Jenkinsfiles (API and mobile), merged and building green.
- Architecture diagram and rollback runbook (PDF or one page).
- A recorded or live 10-minute walkthrough demonstrating at least one gate actively blocking a bad change.

ASSESSMENT

Criterion	Points
All prior labs' gates present, correctly ordered, and parallelized where independent	30
No hardcoded secrets anywhere in either Jenkinsfile	15
Mobile pipeline builds and signs correctly	20
Pipeline health gate demonstrably blocks a deploy live	15
Runbook is specific and actually executable, not generic	10
Live walkthrough is clear and matches the actual pipeline behavior	10